

Demand Forecasting at Scale

ExpresSo NB Hackathon - คู่มืออ่านโจทย์ภาษาไทยแบบละเอียด

สรุปครบทุกหน้า + จุดต้องทำ + จุดห้ามพลาด + Checklist ก่อนเริ่มและก่อน Submit

เอกสารนี้อ้างอิงจาก Hackathon Kickoff Deck v2 ที่ผู้ใช้อัปโหลดในบทสนทนา

Forecast window	2024-11-01 ถึง 2024-12-31
Granularity	store × category × date × horizon
Submission rows	25,620 rows
Metric	MAE - Lower is better
Deadline	2026-05-15 08:00 Asia/Bangkok

สารบัญอ่านเร็ว

- 1) Executive Summary: โจทย์นี้คืออะไร
- 2) สรุปครบทุกหน้าใน Deck แบบไล่สไลด์
- 3) Business Problem: Forecast ใช้กับ Inventory และ Order อย่างไร
- 4) Target: units_sold ต้องสร้างจากอะไร
- 5) Forecast Granularity และจำนวน Row ที่ต้องส่ง
- 6) Horizon 1d / 7d / 1m ใช้ต่างกันอย่างไร
- 7) Dataset Train/Test และตารางทั้งหมด
- 8) จุดห้ามทำและ Data Leakage
- 9) Metric, Leaderboard, Submission Format, Compute Limit
- 10) แนวทางเริ่มทำงานจริง
- 11) Checklist ก่อนเริ่มและก่อน Submit
- 12) One-page Cheat Sheet

แก่นของโจทย์: ต้องทำนาย units_sold ระดับ store × category × forecast_date × horizon ให้ครบ 25,620 rows โดย target ต้องมาจาก TRANSACTION + ORDER + PRODUCT เท่านั้น และต้องระวัง leakage ตาม horizon

ส่วนที่ 1: Executive Summary - โจทย์นี้คืออะไร

โจทย์ Demand Forecasting at Scale ของ ExpresSo NB เป็นโจทย์พยากรณ์ยอดขายสำหรับร้านกาแฟขนาดใหญ่ โดยใช้ข้อมูลย้อนหลังเพื่อทำนายจำนวนหน่วยที่ขายได้ในอนาคตในระดับร้าน สินค้าหมวดหมู่ วัน และ horizon การพยากรณ์ผลลัพธ์สุดท้ายที่ต้องส่งคือไฟล์ submission ที่มี prediction ครบทุกแถวตาม sample_submission.csv โดยแต่ละแถวมี id และ units_sold_predicted

หัวข้อ	รายละเอียด
Objective	Forecast units sold ของแต่ละ store × category × day ที่ 3 horizons
Target	units_sold จาก TRANSACTION ✕ ORDER ✕ PRODUCT แล้ว groupby store_id, category, date
Forecast window	2024-11-01 ถึง 2024-12-31 รวม 61 วัน
Stores	20 representative stores
Categories	7 product categories จาก PRODUCT.category
Horizons	1d, 7d, 1m
Submission rows	$20 \times 7 \times 61 \times 3 = 25,620$ rows
Metric	MAE - lower is better

ประโยคง่าย: ทำโมเดลให้ตอบว่า “ร้านไหน หมวดไหน วันที่ไหน horizon ไหน จะขายได้กี่หน่วย” และส่งครบทุก combination ห้ามหายแม้แต่แถวเดียว

ส่วนที่ 2: สรุปครบทุกหน้าใน Deck แบบไล่สไลด์

ส่วนนี้ไล่ตามสไลด์ 01-18 เพื่อให้เห็นว่าแต่ละหน้าบอกอะไรและต้องทำอะไรบ้าง โดยไม่ต้องประเด็นสำคัญของโจทย์ออก

01 / 18 - หน้าปกและกรอบการแข่งขัน

โจทย์คือ Demand Forecasting at Scale สำหรับ Expresso NB ให้ forecast 20 stores × 7 categories × 61 days × 3 horizons และมี deadline วันที่ 2026-05-15 เวลา 08:00 ICT ทีมละ 4-6 คน

02 / 18 - About Us

ผู้จัด/พันธมิตรเป็น corporate innovation arm ของ PTT Group เน้น Corporate VC, Corporate VB, Climate Tech และ AI เพื่อแก้โจทย์จริงในเครือ PTT และสร้าง business value

03 / 18 - The Problem

Expresso NB มีเครือข่ายประมาณ 4,400 stores และต้องตัดสินใจ operation ทุกวัน 2 เรื่อง: inventory optimization และ order optimization โดยใน hackathon ใช้ 20 representative stores

04 / 18 - The Task

ทำนาย units sold ราย store × category × day ที่ 3 horizons คือ 1d, 7d, 1m ในช่วง 2024-11-01 ถึง 2024-12-31 รวม 25,620 rows

05 / 18 - Categories

ใช้ category จาก PRODUCT.category ทั้งหมด 7 หมวด: Coffee, Tea, Chocolate & Milk, Juice & Smoothie, Bakery, Savory Bakery, Merchandise ทำนายระดับ category ต่อ store ไม่ใช่ SKU และไม่รวมทุก store

06 / 18 - Gotcha #1: Leakage

ต้องยืนยัน decision day ไม่ใช่ forecast day ทุก feature ต้อง shift อย่างน้อย h วัน เช่น forecast Nov 1: 1d ใช้ถึง Oct 31, 7d ใช้ถึง Oct 25, 30d ใช้ถึง Oct 2

07 / 18 - Test data

test ไม่มี transaction/order/inventory ของช่วงอนาคต ให้ใช้ได้เฉพาะ lookup ที่รู้ล่วงหน้า วิธีใช้ lag ใน test มี 2 ทาง: recursive prediction หรือ train-only cutoff

08 / 18 - train/

train มี 22 เดือน ตั้งแต่ 2023-01-01 ถึง 2024-10-31 รวม 670 วัน มี 9 tables: TRANSACTION, ORDER, INVENTORY, PROMOTION, LOCAL_EVENT, CUSTOMER, DATE_DIM, STORE, PRODUCT

09 / 18 - test/

test เป็น static lookups เฉพาะ Nov-Dec 2024: DATE_DIM, STORE, PRODUCT, PROMOTION, LOCAL_EVENT, sample_submission และห้าม reference test/TRANSACTION, test/ORDER, test/INVENTORY

10 / 18 - The target

target ที่ถูกต้องสร้างจาก TRANSACTION join ORDER join PRODUCT แล้ว groupby store_id, category, date และ sum units_sold ต้องเติมวันที่ไม่มีขายเป็น 0

11 / 18 - Heads-up

data gotchas 5 ข้อ: discount เป็น percent, INVENTORY.units_sold noisy, lookup train/test byte-identical, customer_id null คือ walk-ins, test stockout rate สูงกว่า train

12 / 18 - Stockouts

ทำนาย recorded value เหมือน training data ไม่ต้อง uncensor lost demand ใน stockout days อาจ down-weight stockout rows ด้วย sample_weight ประมาณ 0.3

13 / 18 - Metric

ใช้ MAE เท่านั้นสำหรับ ranking, prediction ต้องไม่ติดลบ ถ้าติดลบจะถูก clip เป็น 0, diagnostics อื่นเช่น RMSE/R2/MAPE ไม่นับคะแนน

14 / 18 - Leaderboards

split ตาม date ไม่ใช่ random: Public = Nov 2024 จำนวน 12,600 rows, Private = Dec 2024 จำนวน 13,020 rows อย่า overfit public

15 / 18 - Format

submission ต้อง match sample_submission.csv exact มี 2 columns คือ id และ units_sold_predicted จำนวน 25,620 rows ห้ามแยก store/category/date/horizon เป็น columns เพิ่ม

16 / 18 - Compute

notebook ต้องรันบน Google Colab free tier: CPU 2 cores, RAM 12 GB, wall-clock $\leq$ 2 hr, GPU optional และห้าม assume ว่ามี

17 / 18 - Rules

deadline 2026-05-15 08:00 Asia/Bangkok, ทีม 4-6 คน, external data ใช้ได้ถ้า public และ cite, pretrained models ต้องโหลด public ได้, ส่ง reproducible notebook

18 / 18 - Q&A

อ่าน Brief และ Competition_Description.md เพิ่มเติม แข่งขันที่ Kaggle: super-ai-engineer-season-6-coffee-chain-hackathon และ deck ย้ำว่า read the data first

ส่วนที่ 3: Business Problem - Forecast ใช้กับ Inventory และ Order อย่างไร

Deck ระบุว่าธุรกิจมีบริบทประมาณ 4,400 stores และ forecast ขับเคลื่อน decision สำคัญ 2 workflow ทุกวัน ได้แก่ inventory optimization และ order optimization ใน hackathon นี้ใช้ 20 stores เป็นตัวแทนของเครือข่ายใหญ่

3.1 Inventory Optimization

Inventory optimization คือการใช้ forecast เพื่อตัดสินใจว่าแต่ละร้านควรมีสินค้าแต่ละ category เท่าไร เพื่อไม่ให้ของขาดหรือของเกิน

หัวข้อ	รายละเอียด
ถ้า forecast ต่ำเกินไป	เสี่ยง understocking, stockout, เสียโอกาสขาย, ลูกค้าซื้อไม่ได้
ถ้า forecast สูงเกินไป	เสี่ยง overstocking, waste, ต้นทุนจม โดยเฉพาะของสดหรือของมีอายุสั้น
forecast ดี	ช่วยเตรียม stock ให้พอดีกับ demand ของแต่ละ store และ category

3.2 Order Optimization

Order optimization คือการใช้ forecast เพื่อวางแผนสั่งของจาก supplier หรือคลังสินค้าให้ตรง cadence และ quantity ที่เหมาะสม เช่น การเติมของวันถัดไป การสั่งรอบสัปดาห์ และการวางแผนรายเดือน

หัวข้อ	รายละเอียด
1d forecast	ใช้เตรียมของวันถัดไปหรือเติม stock ระยะสั้น
7d forecast	ใช้วางแผน supplier ordering รายสัปดาห์
1m forecast	ใช้วางแผนรายเดือน seasonal planning และภาพรวม supply chain

ส่วนที่ 4: Target - units_sold ต้องสร้างจากอะไร

Target ที่ต้องทำนายคือจำนวนหน่วยที่ขายได้จริงในระดับ store_id, category, date โดยต้องสร้างจากยอดขาย line-level ใน TRANSACTION แล้วเชื่อมข้อมูล order และ product เพื่อรู้ร้าน วันที่ และ category

```
target = (TRANSACTION
    .merge(ORDER[["order_id", "store_id", "date"]], on="order_id")
    .merge(PRODUCT[["product_id", "category"]], on="product_id")
    .groupby(["store_id", "category", "date"])["units_sold"]
    .sum())
```

หัวข้อ	รายละเอียด
TRANSACTION	ใช้ column units_sold ระดับ line item และ product_id
ORDER	ใช้ order_id เพื่อดึง store_id และ date ของ order
PRODUCT	ใช้ product_id เพื่อดึง category
Aggregation	groupby store_id + category + date แล้ว sum units_sold
Zero days	วันที่ไม่มี sales ต้องถือเป็น 0 ไม่ใช่ missing

ห้ามผิดเด็ดขาด: ห้ามใช้ INVENTORY.units_sold เป็น target เพราะ deck ระบุว่า noisy และ match กับ transaction ระดับ SKU/day

แค่ประมาณ 12% เท่านั้น

ส่วนที่ 5: Forecast Granularity และจำนวน Row ที่ต้องส่ง

Granularity ที่ถูกต้องคือ store x category x forecast_date x horizon หมายความว่าทุก row ใน submission แทน prediction หนึ่งค่าเท่านั้น

หัวข้อ	รายละเอียด
store_id	ร้านแต่ละสาขา รวม 20 stores
category	หมวดสินค้า รวม 7 categories
forecast_date	วันที่ต้องทำนาย 2024-11-01 ถึง 2024-12-31
horizon	ระยะล่วงหน้า 1d, 7d, 1m
prediction	units_sold_predicted เป็น float ≥ 0

สูตรจำนวนแถวที่ต้องส่ง:

$20 \text{ stores} \times 7 \text{ categories} \times 61 \text{ days} \times 3 \text{ horizons} = 25,620 \text{ rows}$

ข้อควรจำ: Forecast at category level per store. ไม่ใช่ SKU level และไม่ aggregatedข้าม stores

ส่วนที่ 6: Categories ทั้ง 7 หมวด

ลำดับ	Category จาก PRODUCT.category
1	Coffee
2	Tea
3	Chocolate & Milk
4	Juice & Smoothie
5	Bakery
6	Savory Bakery
7	Merchandise

Category ต้องมาจาก PRODUCT.category และ target ต้อง aggregate จาก product_id ขึ้นมาเป็น category

ส่วนที่ 7: Horizon 1d / 7d / 1m ใช้ต่างกันอย่างไร

Horizon	Business Use Case
1d - next-day	ใช้สำหรับ same-day prep และ last-minute stock เช่น เตรียมของพาร์ตนี้ เติม stock ก่อนร้านเปิด
7d - week-ahead	ใช้สำหรับ weekly supplier ordering cycle เช่น สั่งวัตถุดิบรายสัปดาห์
1m - month-ahead	ใช้สำหรับ monthly planning และ seasonal positioning เช่น เตรียม stock เทศกาล วาง supply plan รายเดือน

ผลต่อ feature engineering: ยิ่ง horizon ยาว ยิ่งต้องถอยข้อมูล historical feature ให้ไกลขึ้น เช่น 1m ต้องไม่ใช่ข้อมูลใน 30 วันก่อน

ส่วนที่ 8: Dataset Train/Test และตารางทั้งหมด

8.1 Train Dataset

train period คือ 2023-01-01 ถึง 2024-10-31 รวม 22 เดือน หรือ 670 วัน

Table ใน train/	ใช้ทำอะไร
TRANSACTION	Line-level sales เช่น units, revenue, discount
ORDER	Order header เช่น store, date, hour, customer, payment
INVENTORY	Daily stock state และ is_stockout
PROMOTION	Campaign schedule เช่น promo type, discount, BOGO, loyalty, channel flags
LOCAL_EVENT	Local events เช่น market, cultural, sports, concert และอื่น ๆ
CUSTOMER	Member registry
DATE_DIM	Calendar เช่น holiday, payday, season flags
STORE	Store profiles ทั้ง 20 stores เช่น type, capacity, hours
PRODUCT	Catalog รวมถึง category

8.2 Test Dataset

test period คือ Nov-Dec 2024 แต่มีเฉพาะ static lookup และข้อมูลที่อยู่ล่วงหน้า ไม่มี transaction/order/inventory ของอนาคต

Table ใน test/	ใช้ทำอะไร
DATE_DIM.csv	calendar สำหรับช่วง forecast
STORE.csv	store attributes ที่ไม่เปลี่ยน
PRODUCT.csv	catalog ที่ไม่เปลี่ยน
PROMOTION.csv	planned promotions สำหรับ Nov-Dec 2024
LOCAL_EVENT.csv	scheduled events สำหรับ Nov-Dec 2024
sample_submission.csv	template submission จำนวน 25,620 rows ค่าเริ่มต้น prediction = 0

ห้าม reference: ห้ามอ่านหรือ reference test/TRANSACTION.csv, test/ORDER.csv, test/INVENTORY.csv เพราะไฟล์เหล่านี้ถูก omit ตั้งใจ ถ้า notebook reference จะ invalidated

ส่วนที่ 9: จุดห้ามทำและ Data Leakage

9.1 Decision day ไม่ใช่ Forecast day

แนวคิดสำคัญของโจทย์นี้คือเวลาสร้าง feature ต้องยืนอยู่บนวันที่ตัดสินใจหรือวันที่รันโมเดล ไม่ใช่วันที่เรากำลังทำนาย

หัวข้อ	รายละเอียด
Decision day	วันที่เรารันโมเดลและ generate prediction
Forecast date	วันที่อนาคตที่เราจะทำนายยอดขาย

Rule	ทุก feature ต้อง shift อย่างน้อย h วันตาม horizon
ตัวอย่าง	ข้อมูลล่าสุดที่ใช้ได้
1d forecast for 2024-11-01	ใช้ข้อมูล historical ได้ถึงไม่เกิน 2024-10-31
7d forecast for 2024-11-01	ใช้ข้อมูล historical ได้ถึงไม่เกิน 2024-10-25
1m / 30d forecast for 2024-11-01	ใช้ข้อมูล historical ได้ถึงไม่เกิน 2024-10-02

9.2 ตัวอย่าง Leakage ที่ deck ย้ำ

- lag_1 ใน 7d model = leakage เพราะใช้ข้อมูลใกล้ forecast date เกินไป
- 7-day rolling mean ของ last week ใน 30d model = leakage เพราะ last week อยู่ใน forbidden window
- rolling window สำหรับ 1m ต้องเริ่มอย่างน้อย 30 วันก่อน forecast date
- judges จะตรวจ code review เรื่อง leakage

9.3 วิธีใช้ lag ใน Test Window

ทางเลือก	ความหมาย
Option A: Recursive prediction	ใช้ prediction ของตัวเอง feed กลับไปเป็น lag สำหรับวันถัด ๆ ไป
Option B: Train-only cutoff	ใช้เฉพาะข้อมูล $\leq$ 2024-10-31 สำหรับทุก forecast

ส่วนที่ 10: Data Integrity Gotchas ที่ต้องจำ

Gotcha	คำอธิบาย
discount_applied เป็น percent	ค่าอยู่ในช่วง 0-100 ไม่ใช่ fraction ต้องใช้สูตร $revenue \approx base_price \times units \times (1 - discount/100)$
INVENTORY.units_sold noisy	มีเพียงประมาณ 12% ที่ match TXN ระดับ SKU/day grain ห้ามใช้เป็น target
PROMOTION, LOCAL_EVENT, DATE_DIM	train และ test versions เป็น byte-identical lookups
customer_id = null	ประมาณ 60% ของ orders เป็น null เพราะเป็น walk-ins เป็น structural ไม่ใช่ missing ผิดปกติ
stockout drift	Test stockout rate 8.20% vs train 6.58% มี slight drift ต้องระวังถ้าใช้ stockout-aware model

10.1 Stockouts: ทำนาย recorded value ไม่ต้อง uncensor demand

บน stockout days ยอดขายที่ record จะเป็น truncated quantity เพราะขายหยุดเมื่อ stock = 0 deck ระบุว่าไม่ต้อง recover lost demand เพราะ grader เทียบ prediction กับ recorded historical values

Tip จาก deck: ลอง down-weight stockout rows เช่น sample_weight $\approx$ 0.3 เพื่อไม่ให้โมเดลถูกดึงเข้าหา truncated values มากเกินไป

ส่วนที่ 11: Metric, Leaderboard, Submission Format, Compute Limit

11.1 Metric

Metric ที่ใช้ ranking คือ MAE เท่านั้น lower is better

$$MAE = (1 / N) \cdot \sum |\hat{y} - y|$$

- ประเมินทุกแถวใน store × category × forecast_date × horizon
- prediction ต้อง non-negative ถ้าติดลบจะถูก clipped เป็น 0
- RMSE, R², MAPE ใช้เป็น diagnostics ได้ แต่ไม่ใช่จัดอันดับ

11.2 Leaderboard Split

หัวข้อ	รายละเอียด
Public leaderboard	Nov 2024: 2024-11-01 ถึง 2024-11-30 จำนวน 12,600 rows
Private leaderboard	Dec 2024: 2024-12-01 ถึง 2024-12-31 จำนวน 13,020 rows
Split style	split ตาม date ไม่ใช่ random
ความเสี่ยง	overfit Public Nov อาจไม่ช่วย Private Dec เพราะ late-period drift

11.3 Submission Format

Column / Rule	รายละเอียด
id	string รูปแบบ {store_id}_{category}_{forecast_date}_{horizon} เช่น 1_Bakery_2024-11-01_1d
units_sold_predicted	float ≥ 0 เป็นค่าพยากรณ์
row count	25,620 rows
columns เพิ่ม	ห้าม include store/category/date/horizon เป็น separate columns

11.4 Compute Limit

หัวข้อ	รายละเอียด
CPU	2 cores
RAM	12 GB
Wall-clock	≤ 2 hr
GPU	optional ใช้ได้ถ้ามี แต่ห้าม assume
หากเกิน limit	judge rerun แล้วเกิน budget = disqualification

11.5 Rules

หัวข้อ	รายละเอียด
Deadline	2026-05-15, 08:00 Asia/Bangkok
Team size	4 ถึง 6 members
External data	Allowed ถ้า publicly accessible และ cited

Pre-trained models	Allowed แต่ต้อง downloadable จาก public source
Code submission	Required: reproducible notebook บน Colab free tier
Submissions per team	4 per day และ highest score counts

ส่วนที่ 12: แนวทางเริ่มทำงานจริง

ลำดับการทำงานที่แนะนำเพื่อให้ไม่หลงโจทย์และลดความเสี่ยงเรื่อง leakage

1. เปิด sample_submission.csv ก่อน เพื่อรู้ exact row/id ที่ต้องส่ง
2. สร้าง canonical target จาก TRANSACTION + ORDER + PRODUCT
3. สร้าง full grid ของ train: store x category x date แล้วเติม zero days = 0
4. สร้าง calendar features จาก DATE_DIM
5. join STORE และ PRODUCT เพื่อใช้ static attributes
6. สร้าง promotion และ local event features จากข้อมูลที่มีทั้ง train/test
7. สร้าง lag/rolling features แบบ shift ตาม horizon อย่างเข้มงวด
8. ทำ baseline ง่าย ๆ เช่น historical mean หรือ rolling mean ที่ไม่ leak
9. ทำ time-based validation ไม่ใช่ random split
10. วัด MAE แยกตาม horizon, category, store เพื่อดูจุดอ่อน
11. เทรน model เช่น LightGBM/CatBoost/XGBoost/RandomForest โดยควบคุม compute
12. generate submission โดย align กับ sample_submission.csv เท่านั้น
13. ตรวจสอบ row count, column, id, NaN, negative, duplicate ก่อน submit

หลักคิดสำคัญ: โมเดลต้องอย่างเดียวไม่พอ ต้อง target ถูก, leakage ไม่มี, validation ใกล้โจทย์จริง, submission format เป๊ะ

ส่วนที่ 13: Checklist ก่อนเริ่มทำงาน

- ☐ เข้าใจว่า target คือ units_sold ไม่ใช่ revenue
- ☐ รู้ว่า target ต้องสร้างจาก TRANSACTION $\times$ ORDER $\times$ PRODUCT
- ☐ รู้ว่าทำนายระดับ store x category x forecast_date x horizon
- ☐ รู้ว่ามี 20 stores และ 7 categories
- ☐ รู้ว่าช่วง forecast คือ 2024-11-01 ถึง 2024-12-31 รวม 61 วัน
- ☐ รู้ว่าต้องส่ง 25,620 rows
- ☐ เตรียม logic เติม zero sales สำหรับวันไม่มี transaction
- ☐ เตรียม validation แบบ time split
- ☐ ตัดสินใจว่าจะใช้ recursive prediction หรือ train-only cutoff สำหรับ test lag

- ☐ ตั้ง rule ว่าทุก lag/rolling feature ต้อง shift ตาม horizon

ส่วนที่ 14: Checklist ก่อน Submit

- ☐ ไฟล์มี 2 columns เท่านั้น: id, units_sold_predicted
- ☐ row count = 25,620
- ☐ id format ตรง {store_id}_{category}_{forecast_date}_{horizon}
- ☐ ไม่มี duplicated id
- ☐ ไม่มี missing id เมื่อเทียบกับ sample_submission.csv
- ☐ prediction เป็นตัวเลขทั้งหมด
- ☐ ไม่มี NaN หรือ inf
- ☐ ไม่มีค่าติดลบ
- ☐ ครบ 20 stores
- ☐ ครบ 7 categories
- ☐ ครบวันที่ 2024-11-01 ถึง 2024-12-31
- ☐ ครบ horizons 1d, 7d, 1m
- ☐ ไม่มีการอ่าน test/TRANSACTION.csv, test/ORDER.csv, test/INVENTORY.csv
- ☐ feature ทุกตัว shift ตาม horizon แล้ว
- ☐ notebook rerun จาก fresh runtime ได้บน Colab free tier
- ☐ runtime ไม่เกิน 2 ชั่วโมง และ RAM ไม่เกิน 12 GB

ส่วนที่ 15: One-page Cheat Sheet

จำให้ได้	คำตอบ
ต้องทำนายอะไร	units_sold_predicted
target มาจาก	TRANSACTION + ORDER + PRODUCT
ห้ามใช้ target จาก	INVENTORY.units_sold
grain	store × category × forecast_date × horizon
forecast window	2024-11-01 ถึง 2024-12-31
row count	25,620
metric	MAE
public/private	Nov = Public, Dec = Private
test ที่มี	DATE_DIM, STORE, PRODUCT, PROMOTION, LOCAL_EVENT, sample_submission
test ที่ไม่มี/ห้าม reference	TRANSACTION, ORDER, INVENTORY

leakage rule	feature ต้อง shift $\geq$ horizon
compute	Colab free tier: 2 cores, 12GB RAM, ≤ 2 hr

สรุปสุดท้าย: ถ้าอ่านเอกสารนี้แล้วเริ่มทำให้เริ่มจาก sample_submission -> สร้าง target canonical -> full grid zero days -> feature แบบไม่ leak -> time validation -> submission format เป๊ะ